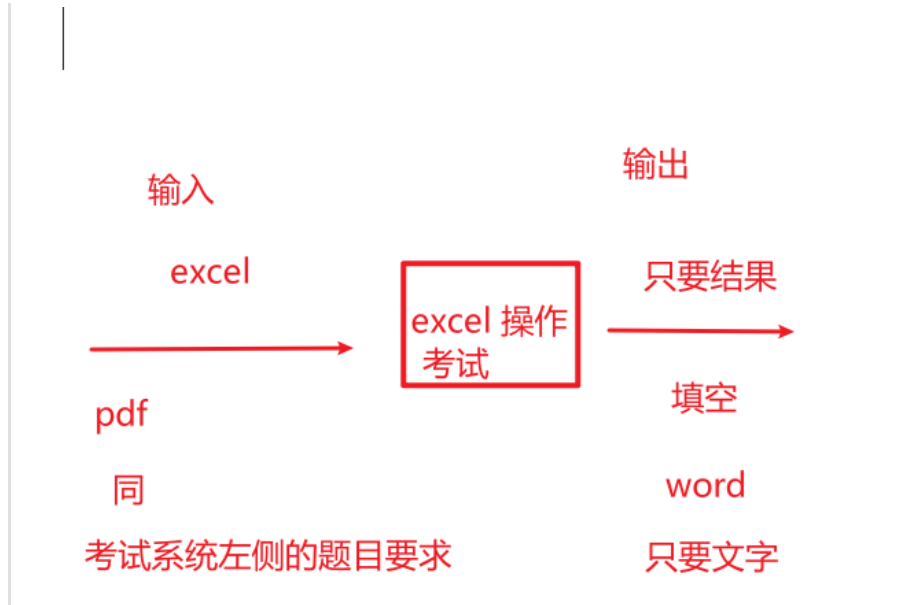


3.1.X

- 题型- excel 操作（9分）+文字题目（6分）
- 只要结果输出，不需要excel 过程，和代码过程



- 考试系统有可能MS office，也可能WPS（基础版免费）
- 3.1.1

测量分评分表

细则编号	配分	评分细则描述	规定或 标称值	结果或 实际值	得分
M1	3	回答用户使用习惯中最常被使用的三个功能：每个功能得1分，本项最高得3分；	根据数据		
M2	3	回答最受欢迎的功能和较少使用的功能：最受欢迎的功能得1分，较少使用的功能每个得1分，本项最高得3分；	根据数据	excel 操作	
M3	3	回答不同功能的平均响应时间：响应时间较长的功能得1分；响应时间适中的功能得1分；响应时间较短的功能得1分；本项最高得3分；	根据数据		9
M4	6	回答优化方向和对应解决方案：每1个正确的优化方向得1分，对应解决方案得1分，本项最多得6分；	根据数据	文字题背诵	6
合计配分	15	合计得分			

功能使用频率：识别最受欢迎的功能和较少使用的功能；

响应时间：考察不同功能的平均响应时间，找出可能的瓶颈。

给出一份在用户使用习惯、功能使用频率和响应时间方面的分析报告，将其保存为 docx 文件，命名为 3.1.1-1.docx。

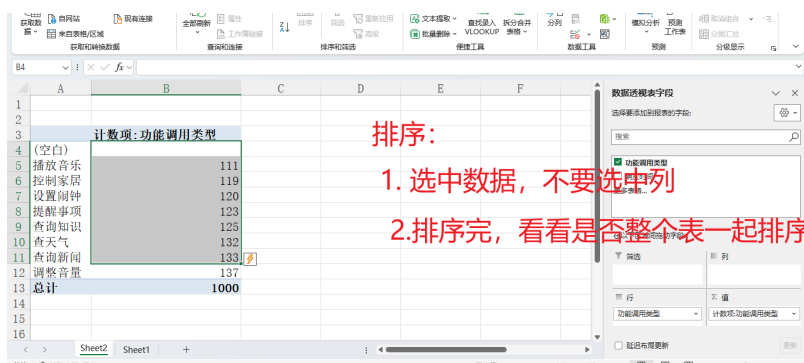
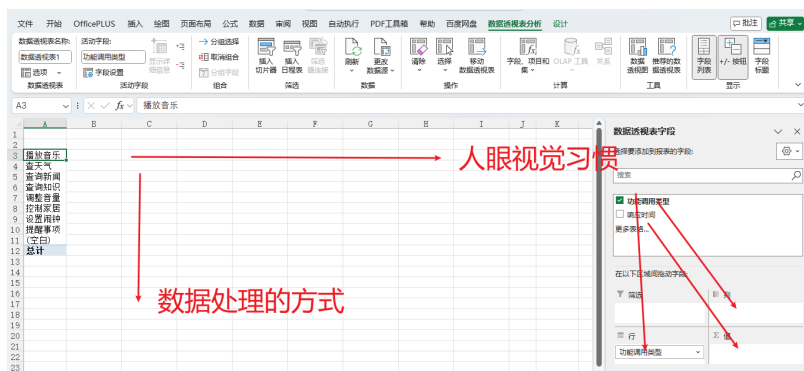
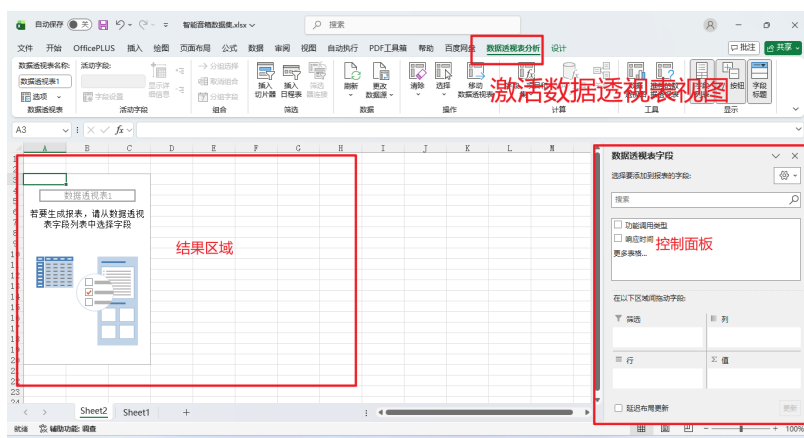
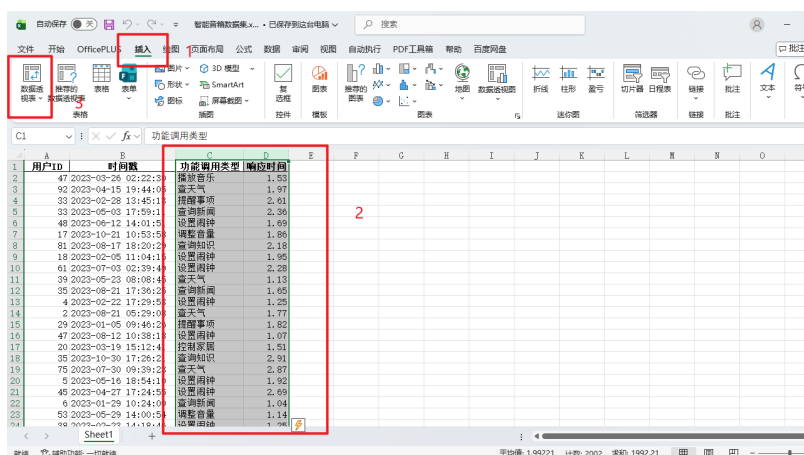
(2) 为了进一步提升用户体验，给出智能音箱产品的 3 个优化方向和对应解决方案，将其

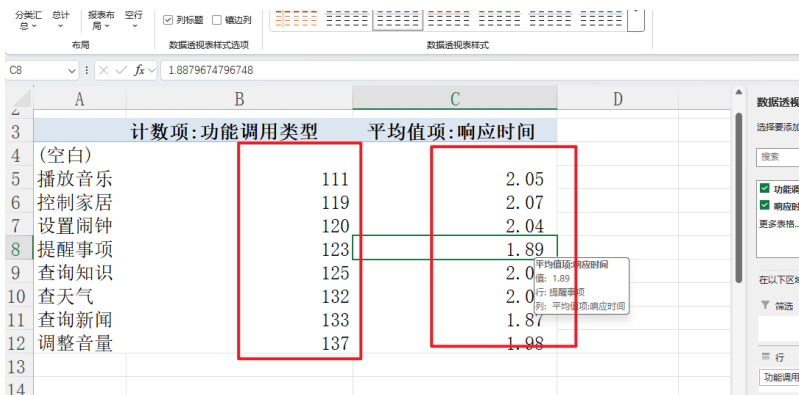
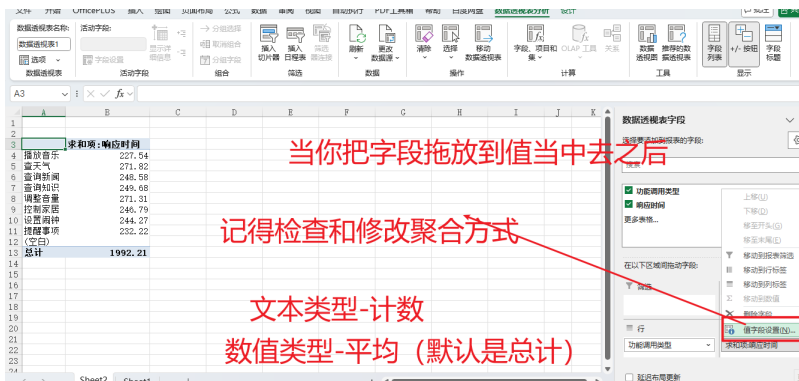
value_counts()

groupby

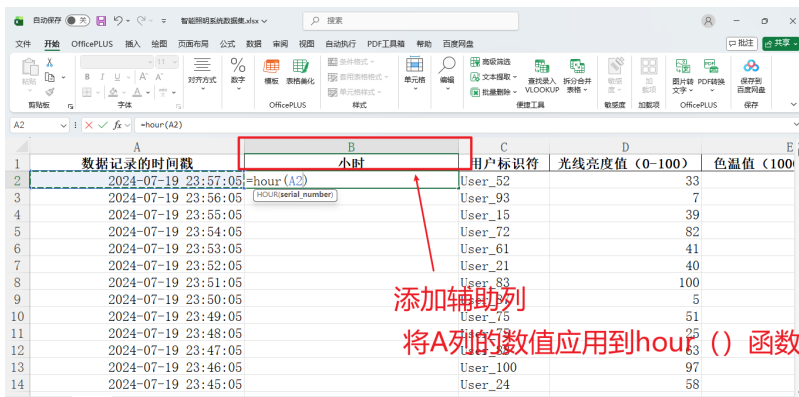
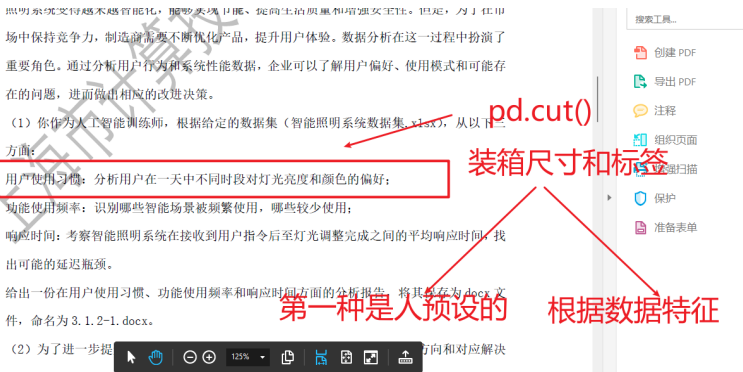
- 数据透视表

- 可以将字段进行分组，然后选择是通过行，还是列的而方式展示，可以通过行，列的交叉，来计算聚合

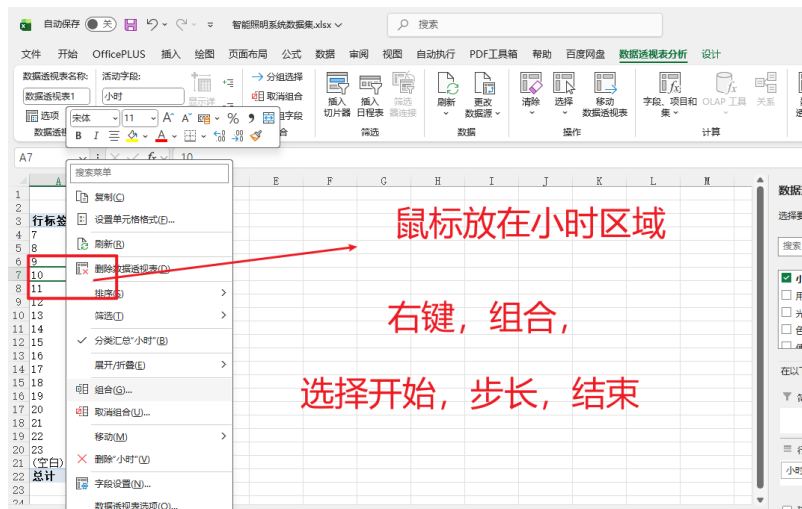
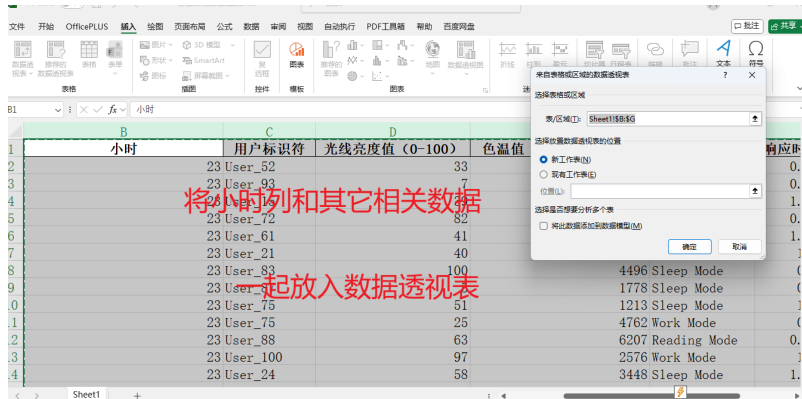
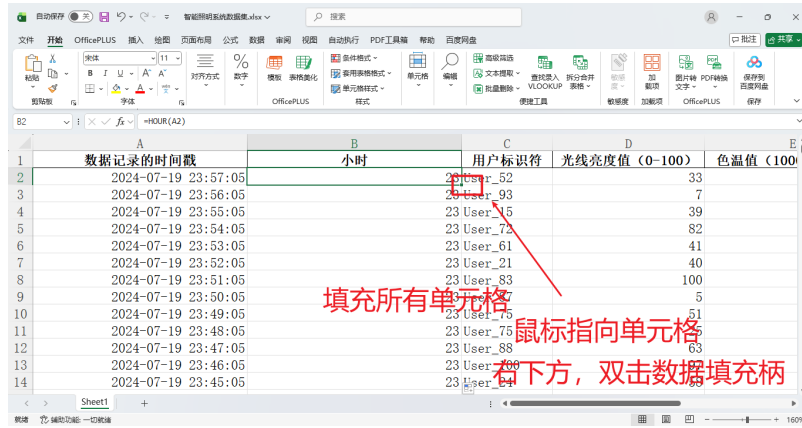
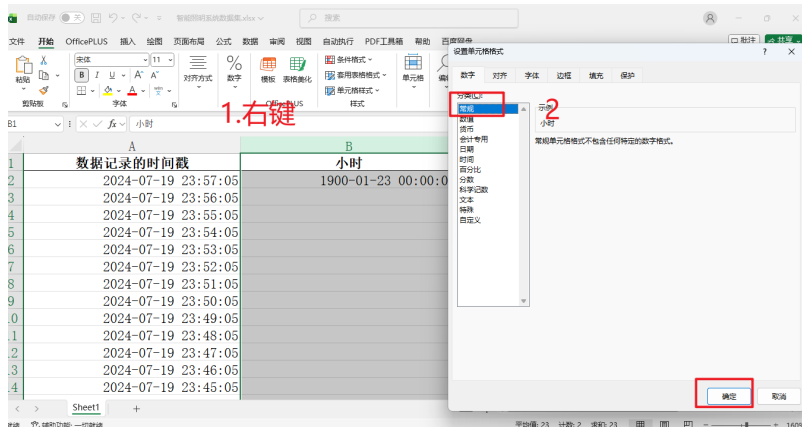


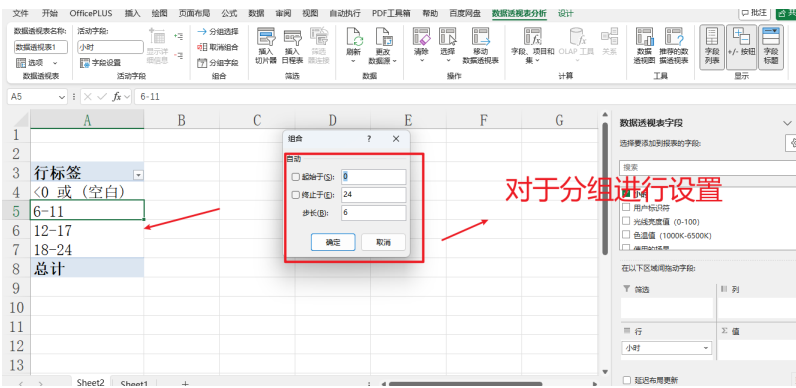


3.1.2



修改单元格格式

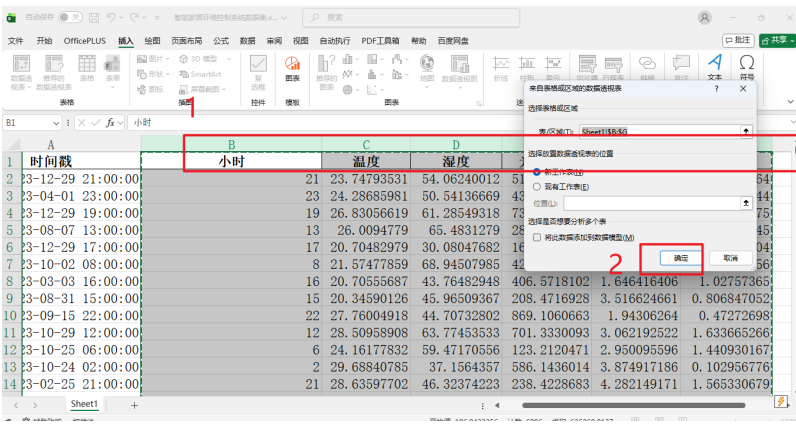
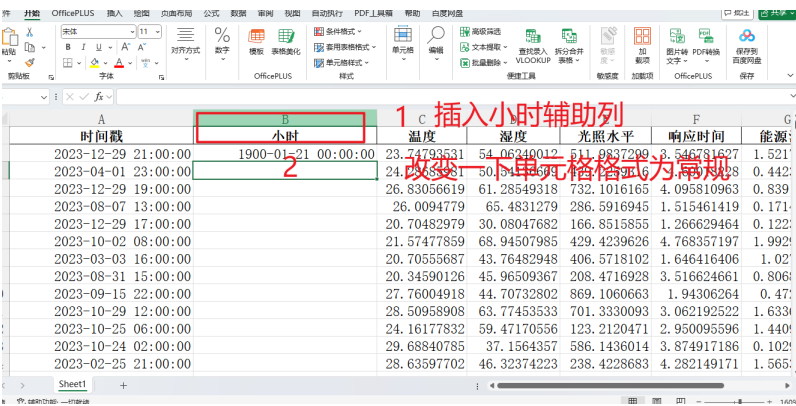


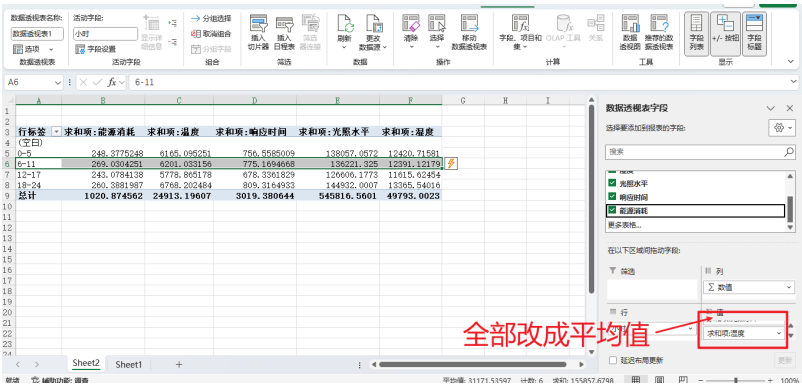
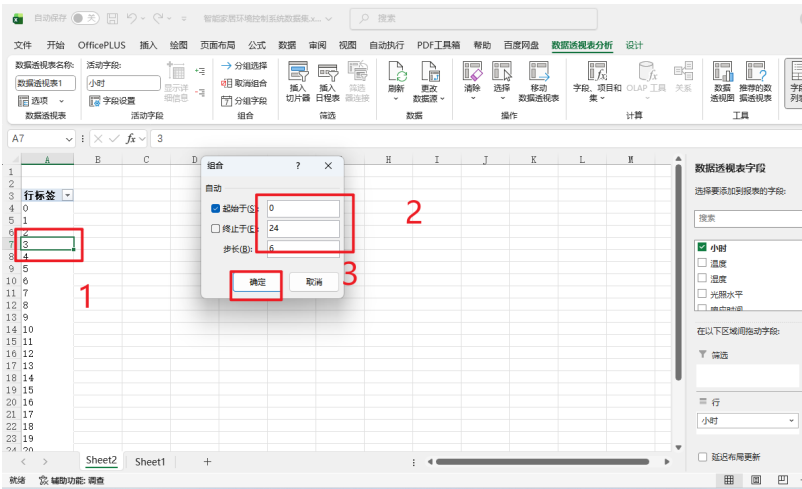


3.1.3

3.1.4

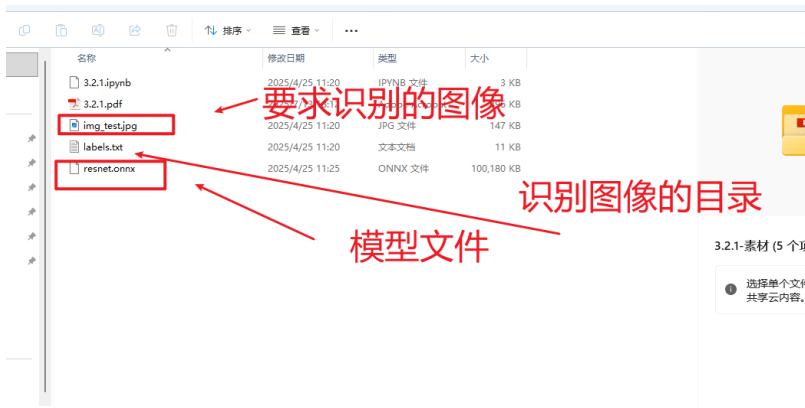
3.1.5





3.2.X

素材解读



图像识别原理

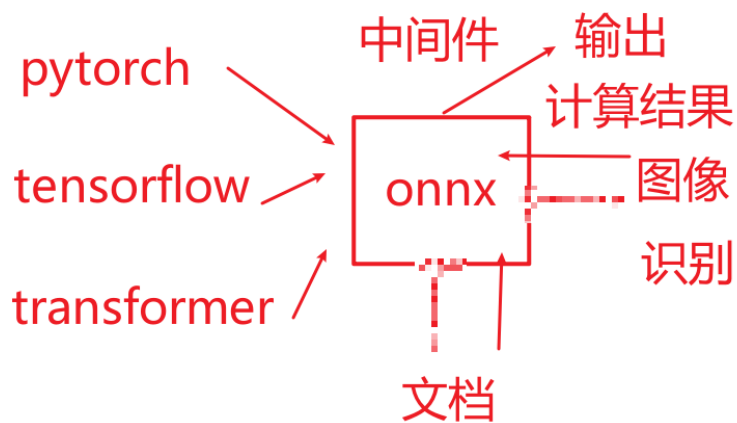
- 通过多维数据来存储和转换图片
- 早期

转换图片

定义模板
早期



- 机器学习
- 深度学习
 - CNN 卷积神经网络使得图像识别进入了高原期
- 深度学习框架的应用-onnx



- 深度学习的门槛
 - 数字类型
 - 标准化
 - 数据必须符合模型维度要求
- 深度学习结果二次计算
 - `scipy.special.softmax()` 将评分转换成相加为100%的百分比[3, 2, 1] 返回[0.6, 0.3, 0.1], 也称归一化
 - `np.argsort()`: 将评分正排序, 返回是索引: [2, 1, 3] 返回: [1第二个索引/最小, python 索引从0开始, 0, 2]
 - `np.argmax()`, 直接返回最大值的索引: [2, 1, 3] 返回2
- 3.2.1
 -

```
[ ]: import onnxruntime as ort
import numpy as np
import scipy.special
from PIL import Image

# 预处理图像
```

打开深度学习模型
读入内存
数据计算
读取图像

```
[1]: import onnxruntime as ort
ort.InferSession
ort.InferenceSession

[ ]:
```

```
# 加载图片 2分
image = Image.open('img_test.jpg').convert('RGB')

# 预处理图片 2分
processed_image = preprocess_image(image)
|
```

计算结果

Jupyter 3.2.1 Last Checkpoint: 7 months ago

Edit View Run Kernel Settings Help Trust

JupyterLab Python 3 (ipykernel)

```
processed_image = processed_image.astype(np.float32)

# 进行图片识别 2分
output = session.run([output_name], {input_name: processed_image})

# 应用 softmax 函数获取概率 2分
probabilities = scipy.special.softmax(output, axis=-1)

# 获取最高的5个概率和对应的类别索引 3分
top5_idx = probabilities.argsort()[::-1]
top5_prob = probabilities[top5_idx]
```

softmax
将一组数据转换成
相加等于100%的概率
[[[3, 2, 1]]]
[0.6, 0.3, 0.1]

```
# 进行图片识别 2分
output = session.run([output_name], {input_name: processed_image})[0]

# 应用 softmax 函数获取概率 2分
probabilities = scipy.special.softmax(output, axis=-1)

# 获取最高的5个概率和对应的类别索引 3分
top5_idx = probabilities.argsort()[::-1]
top5_prob = probabilities[top5_idx]
```

导入的包
备注


```
Kernel Settings Help Trust

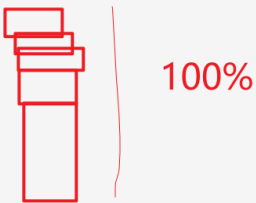
JupyterLab Python 3 (ipykernel)

session.run([output_name], {input_name: processed_image}))[0]

max 函数获取概率 2分
es = scipy.special.softmax(output, axis=-1)

5个概率和对应的类别索引 3分
_____ [-5:] [::-1]
_____

5 predicted classes:")
```



```
jupyter 3.2.1 Last Checkpoint: 7 months ago

Edit View Run Kernel Settings Help Trust

JupyterLab Python 3 (ipykernel)

# 获取最高的5个概率和对应的类别索引 3分
top5_idx = np.argsort(probabilities[0])[-5:] [::-1]
top5_prob = probabilities[0][top5_idx]

# 打印结果
print("Top 5 predicted classes:")
for i in range(5):
    print(f"{i+1}: {labels[top5_idx[i]]} - Probability: {top5_prob[i]}")

Top 5 predicted classes:
1: tusker - Probability: 0.931233286857605
2: Indian elephant - Probability: 0.06785939633846283
3: African elephant - Probability: 0.0009034110116772354
4: gorilla - Probability: 4.78091294553451e-07
5: water buffalo - Probability: 3.9817274455344887e-07

[2]: import onnxruntime as ort
```

3.2.2

```
ort_session = onnxruntime.InferenceSession('mnist.onnx')

# 加载图像 2分
image = Image.open('img_test.png').convert('L') # 转为灰度图

# 图像预处理
image = image.resize((28, 28)) # 调整大小为MNIST模型的输入尺寸2分
image_array = np.array(image, dtype=np.float32) # 转为numpy数组2分
image_array = np.expand_dims(image_array, axis=0) # 添加batch维度2分
image_array = np.expand_dims(image_array, axis=0) # 添加通道维度2分

# 返回模型输入列表 2分

[1]: import numpy as np
np.expand_dims
```

```
# 图像预处理
image = image.resize((28, 28)) # 调整大小为MNIST模型的输入尺寸2分
image_array = np.array(image, dtype=np.float32) # 转为numpy数组2分
image_array = np.expand_dims(image_array, axis=0) # 添加batch维度2分
image_array = np.expand_dims(image_array, axis=0) # 添加通道维度2分

# 返回模型输入列表 2分
```

```

# 图像预处理
image = image.resize((28, 28)) # 调整大小为MNIST模型的输入尺寸2分
image_array = np.array(image, dtype=np.float32) # 转为numpy数组2分
image_array = np.expand_dims(image_array, axis=0) # 添加batch维度2分
image_array = np.expand_dims(image_array, axis=0) # 添加通道维度2分

# 返回模型输入列表 2分
ort_inputs = {ort_session.get_inputs()[0].name: image_array}
# 执行预测 2分
ort_outs = _____(None, ort_inputs)

# 获取预测结果 2分

```

括号不要忘记

3.2.3

- 注意pillow 版本太高报错的解决方法，考试不会出现这个问题

```

# 定义预处理函数，用于将图片转换为模型所需的输入格式
def preprocess(image_path):
    input_shape = (1, 1, 64, 64) # 模型输入期望的形状，这里是 (N, C, H, W), N=batch size, C=channels, H=height, W=width
    img = Image.open(image_path).convert('L') # 打开图像文件并将其转换为灰度图 1分
    img = img.resize((64, 64), Image.ANTIALIAS) # 调整图像大小到模型输入所需的尺寸 1分
    img_data = np.array(img, dtype=np.float32) # 将PIL图像对象转换为numpy数组，并确保数据类型是float32
    # 调整数组的形状以匹配模型输入的形状
    img_data = np.expand_dims(img_data, axis=0) # 添加 batch 维度
    img_data = np.expand_dims(img_data, axis=0) # 添加 channel 维度
    assert img_data.shape == input_shape, f'Expected shape {input_shape}, but got {img_data.shape}' # 确保最终
    return img_data # 返回预处理后的图像数据

```

pillow 从10.0 开始发现bug

如果pillow 版本太高，需要替换
LANCZOS 代替 ANTIALIAS

修复锯齿

```

def preprocess(image_path):
    input_shape = (1, 1, 64, 64) # 模型输入期望的形状，这里是 (N, C, H, W), N=batch size, C=channels, H=height, W=width
    img = Image.open(image_path).convert('L') # 打开图像文件并将其转换为灰度图 1分
    img = img.resize((64, 64), Image.LANCZOS) # 调整图像大小到模型输入所需的尺寸 1分
    img_data = np.array(img, dtype=np.float32) # 将PIL图像对象转换为numpy数组，并确保数据类型是float32
    # 调整数组的形状以匹配模型输入的形状
    img_data = np.expand_dims(img_data, axis=0) # 添加 batch 维度
    img_data = np.expand_dims(img_data, axis=0) # 添加 channel 维度
    assert img_data.shape == input_shape, f'Expected shape {input_shape}, but got {img_data.shape}' # 确保最终
    return img_data # 返回预处理后的图像数据

# 定义情感类别与数字标签的映射表 3分
emotion_table = {'neutral':0, 'happiness':1, 'surprise':2,
'sadness':3, 'anger':4, 'disgust':5, 'fear':6, 'contempt':7}

# 加载模型 3分
ort_session = ort.InferenceSession('emotion-ferplus.onnx') # 使用onnxruntime创建一个会话，用于加载并运行模型

# 加载本地图片并进行预处理 3分
input_data = preprocess('img_test.png')

# 准备输入数据，确保其符合模型输入的要求
ort_inputs = {ort_session.get_inputs()[0].name: input_data} # ort_session.get_inputs()[0].name 是获取模型的第一个输入的名字

# 运行模型，进行预测 3分
ort_outs = ort_session.run(None, ort_inputs)

# 解码模型输出，找到预测概率最高的情感类别 3分

```

Pillow 读取图片，在函数里面已经完成了

```

# 准备输入数据，确保其符合模型输入的要求
ort_inputs = {ort_session.get_inputs()[0].name: input_data} # ort_

# 运行模型，进行预测 3分
ort_outs = ort_session.run(None, ort_inputs)

# 解码模型输出，找到预测概率最高的情感类别 3分

```

```

ort_outs = ort_session.run(None, ort_inputs)

```

找类别

```

# 解码模型输出，找到预测概率最高的情感类别 3分
predicted_label = _____(ort_outs[0])

```

不是计算概率百分比

```

# 根据预测的标签找到对应的情感名称 3分
predicted_emotion = _____[predicted_label]

```

```
# 解码模型输出，找到预测概率最高的情感类别 3分
predicted_label = np.argmax(ort_outs[0])

# 根据预测的标签找到对应的情感名称 3分
predicted_emotion = list(emotion_table.keys())[predicted_label]

# 输出预测的情感
print(f"Predicted emotion: {predicted_emotion}")
```

keys()
只要键(surprise)

values()
只要后面的值

3.2.4

```
# 加载图片 2分
image = Image.open('flower_test.png').convert('RGB')

# 预处理图片 2分
processed_image = preprocess_image()

# 确保输入数据是 float32 类型
```

```
# 应用 softmax 函数获取识别分类后的准确率 2分
accuracy = _____(output, axis=-1)

# 获取预测的类别索引
predicted_idx = _____
```

3.2.5

```
picked_box_probs[:, 3] *= height
return picked_box_probs[:, :4].astype(np.int32), np.array(picked_

# 从标签文件中读取每一行，并去除行首尾的空白字符，得到类别名称列表 2分
class_names = [name.strip() for name in open('voc-model-labels.txt').r

# 创建 ONNX Runtime 的推理会话，用于运行模型进行推理 2分
ort_session = _____('version-RFB-320.onnx')
```



我们用的是这个

[]:

```
cv2.imshow('Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.imwrite('image.jpg', img)
```

```
# 记录开始时间，用于计算模型推理的耗时
time_time = time.time()
# 使用 ONNX Runtime 运行模型，输入图像数据，得到模型输出的置信度和边界框
confidences, boxes = _____(None, {input_name: image})
# 计算并打印模型推理的耗时
print("cost time:{}".format(time.time() - time_time))
# 调用 predict 函数对模型输出的边界框和置信度进行后处理，得到最终的边界框和置信度
boxes, labels, probs = predict(orig_image.shape[1], orig_image.shape[2],
                                confidences, boxes)
# 遍历每个检测到的目标框
for i in range(boxes.shape[0]):
```

(1) 补全该模型的使用交互流程对应的 Python 代码 (3.2.5.ipynb), 实现本地测试图片文件夹 “imgs” 中所有图片的人脸检测, 将运行结果截图保存至 3.2.5-1.jpg 中, 并将检测结果上传图片上传。

自己新建的文件夹

自己新建的文件夹

(2) 在之前使用交互流程基础上，结合实际应用场景，设计在人脸检测系统中使用“version-RFB-320.onnx”模型的一种人机交互优化方案，包括交互界面布局、操作流程等内容，输出设计为 docx 文件，命名为 3.2.5.docx。

中带框的图片

所有结果文件储存在桌面新建的考生文件夹中，文件夹命名为“准考证号+身份证号后六位”。

3. 技能要求

(1) 能确保模型在单一场景下稳定运行;

time: 0.016 系统结果的截屏

6 系统结果的截图